# Studio Nx

Backend(FastAPI/Python) 30 — get_active_user()

2026-10-12

# Backend(FastAPI/Python) 30 — `get_active_user()`

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/auth_service.py` |
| 작업 | P1-05 로그인: 재발급 전 회원 확인 (PRD 01 FR-105, FR-108) |
| 작성 | Claude (AI), 2026-10-07, 사용자 검토 |
| 이전 · 다음 | 29 `authenticate()` · 31 리프레시 쿠키 |

## 1. 하는 일

회원 번호로 회원을 찾아서, **아직 활동 중(`ACTIVE`)**이면 돌려줍니다. 없거나, 정지되었거나, 탈퇴했으면 401로 응답합니다.

| 회원 번호에 해당하는 회원 | 결과 |
|---|---|
| 활동 중 | `User`를 돌려줍니다 |
| 정지(`SUSPENDED`), 탈퇴(`DELETED`) | 401 `UNAUTHORIZED` "Please sign in again." |
| 없음 | 똑같이 401입니다 |

## 2. 언제 쓰나

| 쓰는 곳 | 하는 일 |
|---|---|
| P1-05 `POST /auth/refresh` (33) | 28번으로 리프레시 토큰에서 회원 번호를 꺼낸 뒤, 이 함수로 회원을 확인하고 새 Access Token을 줍니다 |

### 이 함수가 없으면 생기는 일

| 순서 | 일어나는 일 |
|---|---|
| ① | 회원이 자동 로그인(3일 쿠키)으로 로그인합니다 |
| ② | 그날 탈퇴합니다(P1-09). 브라우저에는 리프레시 쿠키가 아직 남아 있습니다 |
| ③ | 다음 날 사이트를 열면 `api.js`가 `/auth/refresh`를 부릅니다. 토큰은 진짜이고 3일 안이라서 28번은 통과합니다 |
| ④ | **이 함수가 없다면** 탈퇴한 회원에게 새 Access Token이 나갑니다. 이 함수가 `DELETED`를 보고 401로 응답하기 때문에, 로그인 화면으로 가게 됩니다 |

정지(관리자가 `SUSPENDED`로 바꿈)도 마찬가지입니다. 바꾼 순간부터 재발급이 막히고, 남아 있는 Access Token(최대 30분)도 06번 `get_current_user()`가 막습니다.

### 06번 `get_current_user()`와 비교

| | 06 `get_current_user()` | 30 `get_active_user()` |
|---|---|---|
| 회원 번호를 얻는 곳 | Access Token (05) | Refresh Token (28) |
| 쓰는 곳 | 로그인이 필요한 모든 API | `/auth/refresh` 하나 |
| 검사 | 없거나 `ACTIVE`가 아니면 401 | 같습니다 |

## 3. 코드

**import** (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import check_password_rules, hash_password, verify_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다 (이번에 새로 추가한 것은 없습니다). 이 함수는 **`Session`, `AppError`, `ErrorCode`, `User`, `UserStatus`**를 사용합니다.

### 함수

```
def get_active_user(db: Session, user_id: int) -> User:
    user = db.get(User, user_id)
    if user is None or user.status != UserStatus.ACTIVE:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    return user
```

## 문법 풀이

| import 문법 | 뜻 |
| --- | --- |
| `from app.models.user import UserStatus` | 08번의 회원 상태 목록입니다 |

| 식 | 뜻 |
| --- | --- |
| `db.get(User, user_id)` | 기본 키로 행 하나를 찾습니다. 없으면 `None`입니다 (06번과 같음) |
| `user is None or …` | 앞이 참이면 뒤를 보지 않습니다. 그래서 `None.status` 오류가 나지 않습니다 |
| `user.status != UserStatus.ACTIVE` | 08번의 `StrEnum`끼리 비교합니다 (06번은 문자열 `"ACTIVE"`와 비교하지만 결과는 같습니다) |

## 4. 흐름

1. 회원 번호로 회원을 찾습니다.
2. 없거나 활동 중이 아니면 401로 응답합니다.
3. 회원을 돌려줍니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 403이 아니라 `UNAUTHORIZED` 401 | `api.js`는 재발급이 실패하면(401) 로그인 화면으로 보냅니다. 정지되거나 탈퇴한 회원도 "다시 로그인"으로 끝납니다. 정지 사실은 로그인(29)을 다시 시도했을 때 비밀번호가 맞으면 403으로 알려 줍니다 |
| 없는 회원, 정지, 탈퇴를 같은 응답으로 처리 | 05번, 06번과 같은 이유입니다. 이유를 알려 주지 않습니다 |
| 06번은 고치지 않음 | 템플릿은 "06번이 이 함수를 불러 써도 된다"고 했지만, 06번은 이미 검토가 끝난 코드이고 검사도 두 줄뿐입니다. 06번을 바꾸면 P1-03 문서와 시험도 다시 봐야 합니다 |

## 6. 시험 (`tests/test_auth_service.py`에 4개를 추가해서 모두 48개, 실제 MySQL로 실행)

| 시험 | 확인한 것 |
|---|---|
| 활동 회원 | 그 회원을 돌려줍니다 |
| 정지, 탈퇴 (2개) | 401 `UNAUTHORIZED` "Please sign in again." |
| 없는 회원 번호 | 401 |

전체 시험 213개가 통과했고, `ruff` 검사도 통과했습니다.